网络安全技术第一次实验：基于DES加密的TCP聊天程序

邹佳衡（2312322）

- 网络安全技术第一次实验：基于DES加密的TCP聊天程序
  - 实验要求
  - 实验环境
  - 实验原理
    - DES加解密
    - TCP通讯协议
    - Socket编程
  - 项目组织方式
  - 代码实现
    - DES加解密
      - 常量和函数的定义
      - 加密过程的实现
    - 基于DES加解密的TCP聊天程序
  - 实验结果
    - 关于DES加解密的验证
    - 聊天室测试
  - 实验总结与反思

实验要求

- 利用Socket编写一个TCP聊天程序
- 通讯内容经过DES加解密

实验环境

Visual Studio 2022, ISO C++ 14

实验原理

DES加解密

DES算法是一种基于Feistel网络结构的对称分组加密标准，它将64位明文划分为左右两部分，经过初始置换后执行16轮包含扩展置换、S盒非线性替换及P盒置换的迭代运算。由于DES加密只能一次加密8个字节，最后剩下的那些位数不足的地方我们采用 PKCS#7 填充方法。

1.初始置换

这是加密的第一步。输入的64位明文数据块会根据一个固定的置换表（IP盒）进行重新排列。这个过程不涉及密钥，纯粹是打乱位的顺序，为后续的迭代做准备。置换后，数据被分为左右两部分，各32位，分别记为 L_0 和 R_0。

2.16轮迭代运算

这是DES算法的核心。它会对 L_0 和 R_0 进行16轮完全相同的运算，每一轮都使用一个不同的48位子密钥（K_0到K_{16}）。

左半部分：直接等于上一轮的右半部分。

$$L_i = R_i$$

右半部分：等于上一轮的左半部分与一个复杂函数 F 的结果进行异或（XOR）运算。

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

3.轮函数F

轮函数F包含四个步骤：

1. **扩展置换**：通过E盒实现，将32位的R_{i-1}扩展到48位。
2. **与子密钥异或**：将扩展后的48位数据与当前轮次的48位子密钥进行异或运算。
3. **S盒置换**：这是DES中唯一的非线性变换，也是安全性的核心。它将48位数据分成8组，每组6位，分别通过8个不同的S盒（替换表）压缩成4位，最终输出32位。
4. **P盒置换**：将S盒输出的32位数据再次根据一个固定的P盒置换表进行重新排列，以实现位的扩散。

4.密钥调度算法

它的核心任务是：将用户输入的 64 位初始密钥（其中 8 位为奇偶校验位），通过一系列变换，生成 16 个互不相同的 48 位子密钥

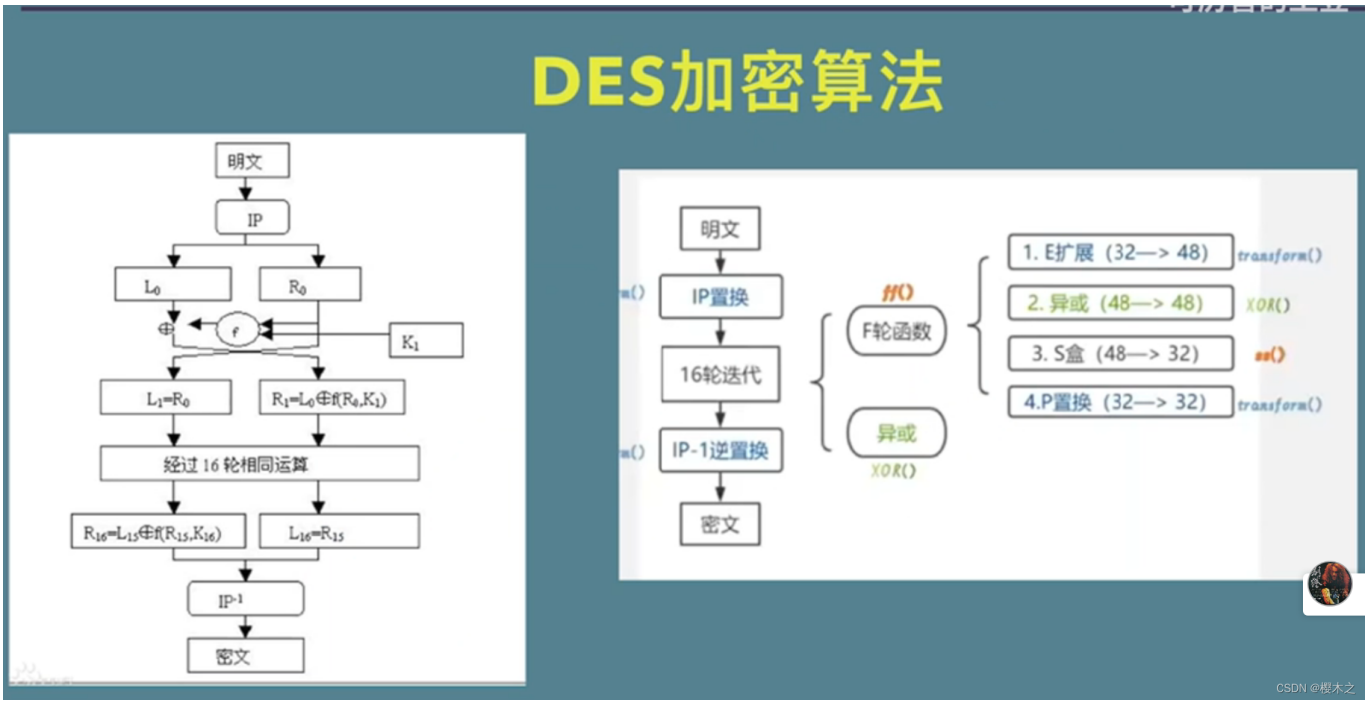
1. **密钥置换选择一**：通过PC-1表将输入的64位密钥丢弃八位，并对剩下的56位密钥进行重排。
2. **密钥分割**：将上一步得到的56位密钥平分成两份，左半部分为 C_0，右半部分为 D_0。
3. **循环左移**：这是生成 16 个不同子密钥的关键。对于 16 轮加密中的每一轮 i（i 从 1 到 16），C_{i-1} 和 D_{i-1} 都会进行一次循环左移，得到新的 C_i 和 D_i 。左移的位数与当前轮次有关。通过这种不规律的移位，确保了每一轮生成的子密钥都是独一无二的，从而增加了算法的安全性。
4. **密钥置换选择二**：在每一轮完成循环左移后，我们会得到一对新的 28 位数据C_I和D_i，将他们合并成新的56位数据块，通过PC-2表挑选出48位，获得本轮的子密钥。

5.逆初始置换

在16轮迭代完成后，会将L_{16}和 R_{16} 进行一次左右交换，组合成 $R_{16}L_{16}$（即右半部分在前，左半部分在后）。然后，对这个64位的数据块进行逆初始置换（IP^{-1}）。IP^{-1}是初始置换IP的逆过程，最终输出的就是64位的密文。

如何解密

DES算法的解密过程利用了Feistel网络结构的对称性，其核心步骤与加密完全相同，唯一的区别在于子密钥的使用顺序相反；即解密时，将64位密文经过初始置换后，在16轮迭代运算中依次逆序使用加密时生成的子密钥，最后经过逆初始置换即可还原出原始明文。



TCP通讯协议

TCP（传输控制协议）是互联网协议套件中的核心传输层协议，它为应用层提供面向连接、可靠、基于字节流的端到端通信服务。在数据传输之前，通信双方必须通过“三次握手”建立一个稳定的逻辑连接。数据传输完毕后，也需要通过“四次挥手”来断开连接，释放系统资源。

1.三次握手

这个过程旨在同步双方的序列号，并确认双方的收发能力都正常。

1. 第一次握手：客户端发送一个SYN（同步）报文，并携带自己的初始序列号，进入“已发送连接请求”状态。
2. 第二次握手：服务器收到SYN报文后，回复一个SYN+ACK（同步+确认）报文，确认客户端的请求，并携带自己的初始序列号，进入“收到连接请求”状态。
3. 第三次握手：客户端收到服务器的回复后，再发送一个ACK（确认）报文，确认服务器的序列号。至此，连接建立成功，双方进入“已建立连接”状态，可以开始传输数据。

2.可靠连接

TCP通过一系列复杂的机制来保证数据不丢失、不重复、且按序到达：

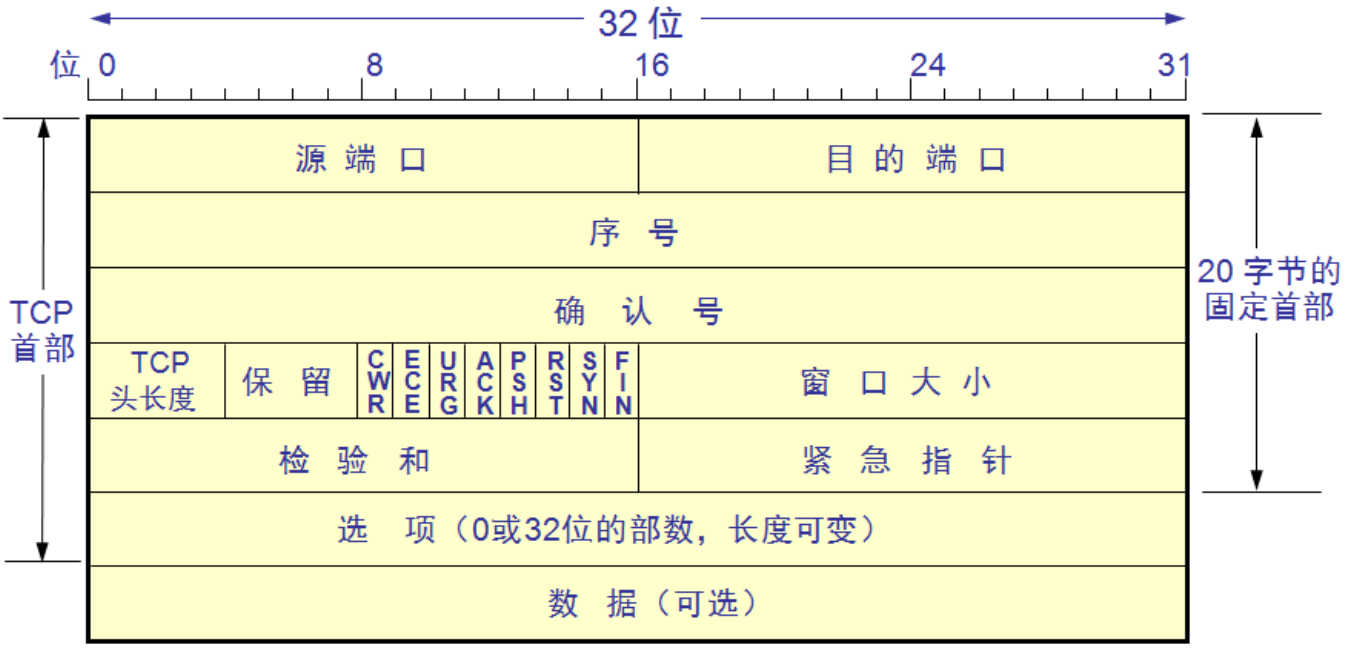
1. 序列号与确认应答：每个发送的数据包都有一个唯一的序列号，接收方收到后会返回一个确认应答（ACK），告知发送方已成功接收。
2. 超时重传：如果发送方在一定时间内没有收到确认应答，就会认为数据包已丢失，并自动重新发送。
3. 校验和：每个数据包都包含一个校验和，用于检测数据在传输过程中是否发生损坏。

3.四次挥手

由于TCP是全双工通信（双方可同时收发数据），断开连接时，双方需要各自独立关闭自己的发送通道。

1. 第一次挥手：主动关闭方（如客户端）发送一个FIN（结束）报文，表示自己没有数据要发送了。
2. 第二次挥手：被动关闭方（如服务器）收到FIN后，回复一个ACK报文进行确认。此时，从客户端到服务器的通道关闭，但服务器可能还有数据要发送给客户端。

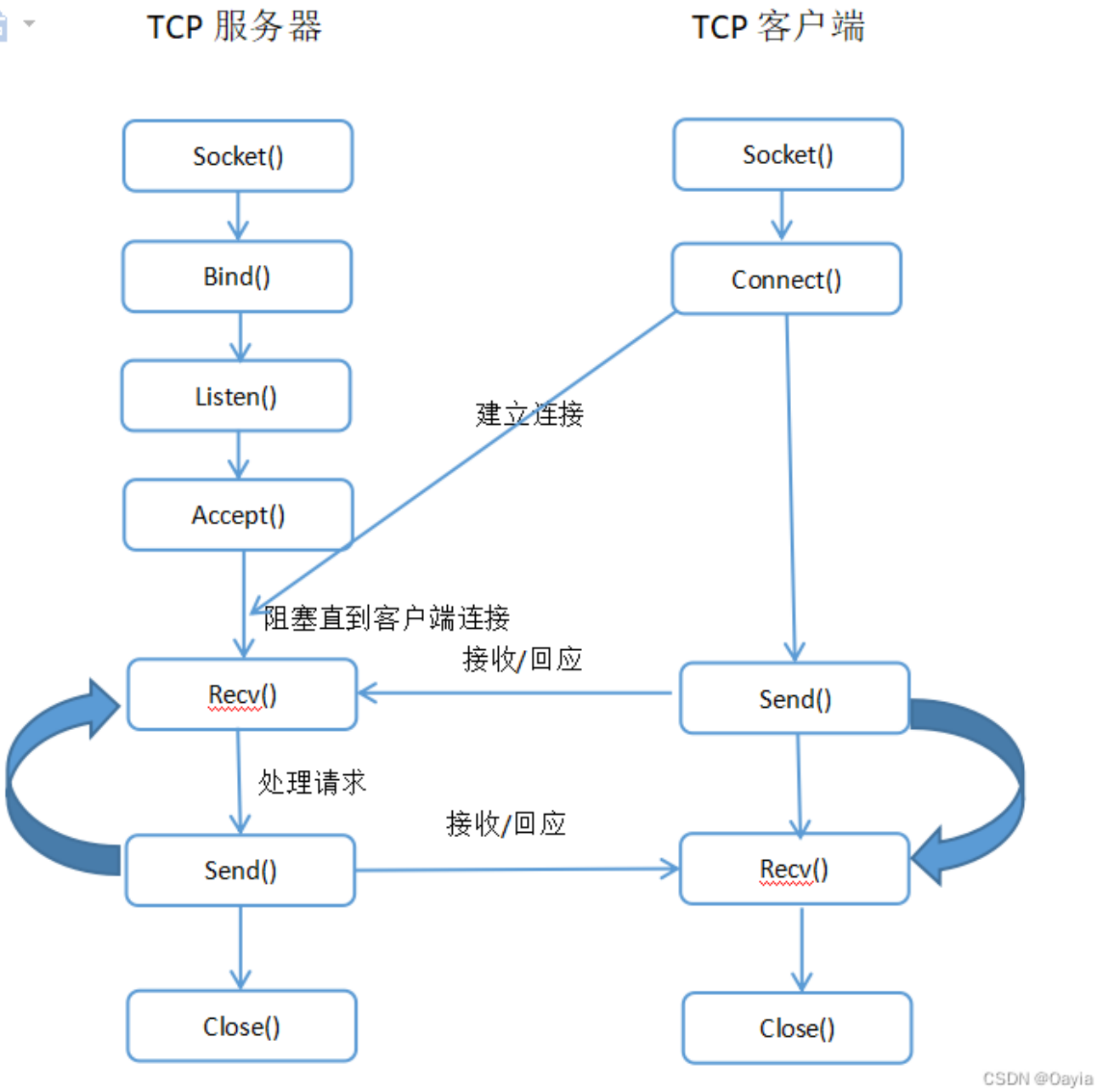
3. 第三次挥手：服务器数据发送完毕后，也发送一个FIN报文，表示自己也没有数据要发送了。
4. 第四次挥手：客户端收到服务器的FIN后，回复最后一个ACK报文进行确认。至此，连接完全断开。



Socket编程

Socket（套接字）编程是网络通信的基石，它为应用程序提供了一套与底层TCP/IP协议栈交互的标准接口。以下是常用的socket函数接口：

1. **socket()**：这是所有网络通信的第一步。它负责创建一个通信端点（即套接字），并返回一个文件描述符（一个非负整数），后续的所有操作都将基于这个描述符进行。
2. **close()**：通信结束后，用于关闭套接字，释放系统资源。
3. **bind()**：将一个本地的 IP 地址和端口号与刚刚创建的套接字绑定在一起。
4. **listen()**：将一个已绑定的套接字转换为“被动监听”状态，准备接受来自客户端的连接请求。
5. **accept()**：这是一个阻塞函数，它会一直等待，直到有一个客户端连接到来。当连接建立后，它会返回一个全新的套接字描述符，专门用于和这个特定的客户端进行通信。原来的监听套接字则继续等待下一个连接。
6. **connect()**：客户端调用此函数，主动向指定的服务器 IP 地址和端口发起连接请求（对于 TCP 而言，这会触发“三次握手”）。连接成功后，这个套接字就可以用于数据收发。
7. **send()**：通过已连接的套接字向对方发送数据。
8. **recv()**：从已连接的套接字接收对方发来的数据。



项目组织方式

在本项目中，总共编写了以下代码文件：

- **client.cpp**：聊天室客户端，用户直接操作的页面，可以连接到服务端IP（默认为127.0.0.1），使用双线程进行交互，主线程读取输入并发送给服务端，子线程持续监听服务端并显示消息。
- **server.cpp**：聊天室服务端，负责管理用户的链接和消息的广播，在8080端口监听用户的连接，接收任一客户端的消息，解密后加上时间戳和昵称，再加密广播给其他所有在线用户。
- **box.h**：声明了 IP 表、逆 IP 表、扩展置换 E 表、P 盒表、PC-1/PC-2 表以及 8 个 S 盒等全局常量。
- **box.cpp**：定义并初始化了 box.h 中所有表的具体数值。
- **DES.h**：声明了加密、解密、字符串与二进制转换等函数原型，这些是外部可以调用的接口。
- **DES.cpp**：实现了 DES 算法的完整逻辑，实现了初始置换(IP)、16 轮迭代置换、逆初始置换(IP^{-1})的全过程。

```
rootDir
├─Client
```

```
│   │  box.cpp
│   │  box.h
│   │  client.cpp
│   │  Client.sln
│   │  Client.vcxproj
│   │  Client.vcxproj.filters
│   │  Client.vcxproj.user
│   │  DES.cpp
│   │  DES.h
│   │
│   ├─Client
│   │   └─x64
│   │       └─Debug
│   │           └─Client.tlog
│   │
│   └─x64
│       └─Debug
│
└─Server
    │  box.cpp
    │  box.h
    │  DES.cpp
    │  DES.h
    │  server.cpp
    │  Server.sln
    │  Server.vcxproj
    │  Server.vcxproj.filters
    │  Server.vcxproj.user
    │
    ├─Server
    │   └─x64
    │       └─Debug
    │           └─Server.tlog
    │
    └─x64
        └─Debug
```

代码实现

DES加解密

常量和函数的定义

```
#pragma once
// box.h
// DES加密算法所需的置换盒和S盒常量声明

#ifndef BOX_H
#define BOX_H

// 初始置换 IP (Initial Permutation)
```

```
extern const int IP[64];

// 逆初始置换 IP^-1 (Inverse Initial Permutation)
extern const int IP_INV[64];

// 扩展置换 E (Expansion Permutation)
extern const int E[48];

// 置换运算 P (Permutation)
extern const int P[32];

// 密钥置换 PC-1 (Permuted Choice 1) - 64位密钥转56位
extern const int PC1[56];

// 密钥置换 PC-2 (Permuted Choice 2) - 56位密钥转48位子密钥
extern const int PC2[48];

// 每轮左移位数
extern const int SHIFT_BITS[16];

// S盒 (S-boxes) - 8个S盒，每个是4x16的矩阵
extern const int S_BOX[8][4][16];

#endif // BOX_H
```

```
// DES.h
// DES加密算法头文件

#ifndef DES_H
#define DES_H

#include <bitset>
#include <string>
#include <vector>

using namespace std;

// 获取全局加密密钥
string getEncryptionKey();

// 辅助函数：字符串与bitset之间的转换
bitset<64> stringToBitset(const string& data);
string bitsetToString(const bitset<64>& bits);

// IP置换函数
bitset<64> ipPermutation(const bitset<64>& input);

// 逆IP置换函数
bitset<64> ipInvPermutation(const bitset<64>& input);

// F函数
```

```
bitset<32> feistelFunction(const bitset<32>& right, const bitset<48>& subKey);

// 子密钥生成函数
vector<bitset<48>> generateSubKeys(const bitset<64>& key);

// 单轮Feistel变换
bitset<64> feistelRound(const bitset<64>& input, const bitset<48>& subKey);

// 完整加密函数
bitset<64> desEncryptBlock(const bitset<64>& plaintext, const bitset<64>& key);

// 完整解密函数
bitset<64> desDecryptBlock(const bitset<64>& ciphertext, const bitset<64>& key);

// 字符串加密函数（支持任意长度）
string desEncryptString(const string& plaintext, const string& key);

// 字符串解密函数（支持任意长度）
string desDecryptString(const string& ciphertext, const string& key);

#endif // DES_H
```

如上面所示，我打算使用 string 类型来储存需要传递的消息，每八个字节为一组进行加密，在加密前先将明文通过 PKCS#7 填充方法填充成8字节的整数倍，再每八个字节为一个单位，转换成 bitset<64> 类型，以便进行更精细的调整

加密过程的实现

1.IP盒置换

```
bitset<64> ipPermutation(const bitset<64>& input) {
    bitset<64> output;

    for (int i = 0; i < 64; i++) {
        int sourceBitIndex = 64 - IP[i];
        output[63 - i] = input[sourceBitIndex];
    }

    return output;
}
```

此函数完成了初始的IP盒置换，简单来说，已知IP盒的第一位是58，就是要将输入的第一位放在输出的第58位，这里的函数里索引看上去不正确，是因为我使用的 bitset 数据类型，它的索引0在右侧，63在左侧。

2.单轮Feistel置换

```
bitset<64> feistelRound(const bitset<64>& input, const bitset<48>& subKey) {
    // 分割成左右两半，各32位
    bitset<32> left;
```

```
    bitset<32> right;

    // 左半部分：高32位（位63-32）
    for (int i = 0; i < 32; i++) {
        left[31 - i] = input[63 - i];
        right[31 - i] = input[31 - i];
    }

    // 保存旧的右半部分
    bitset<32> oldRight = right;

    // 计算新的右半部分：left XOR F(right, subKey)
    bitset<32> newRight = left ^ feistelFunction(oldRight, subKey);

    // 新的左半部分就是旧的右半部分
    bitset<32> newLeft = oldRight;

    // 合并左右两半
    bitset<64> output;
    for (int i = 0; i < 32; i++) {
        output[63 - i] = newLeft[31 - i];
        output[31 - i] = newRight[31 - i];
    }

    return output;
}
```

在完成IP盒置换后，获得的64位结果会立刻进入这一环节，以上是单轮Feistel置换，它接收64位数据和48位子密钥，通过拆分、异或混淆、交换，输出了64位结果。这样的置换需要进行十六次，每次都会得到64位结果，在这之前我们需要先算出并保存16轮的子密钥。

3.密钥调度算法

```
// PC-1置换：将64位密钥转换为56位
bitset<56> pc1Permutation(const bitset<64>& key) {
    bitset<56> output;

    for (int i = 0; i < 56; i++) {
        int sourceBitIndex = 64 - PC1[i];
        output[55 - i] = key[sourceBitIndex];
    }

    return output;
}

// PC-2置换：将56位密钥转换为48位子密钥
bitset<48> pc2Permutation(const bitset<56>& input) {
    bitset<48> output;

    for (int i = 0; i < 48; i++) {
        int sourceBitIndex = 56 - PC2[i];
```

```
        output[47 - i] = input[sourceBitIndex];
    }

    return output;
}

// 循环左移函数
bitset<28> leftShift(const bitset<28>& input, int shift) {
    bitset<28> output;

    for (int i = 0; i < 28; i++) {
        int newPos = (i + shift) % 28;
        output[27 - newPos] = input[27 - i];
    }

    return output;
}

// 生成16轮子密钥
vector<bitset<48>> generateSubKeys(const bitset<64>& key) {
    vector<bitset<48>> subKeys(16);

    // 1. PC-1置换：64位 -> 56位
    bitset<56> permutedKey = pc1Permutation(key);

    // 2. 分成左右两半，各28位
    bitset<28> leftHalf;
    bitset<28> rightHalf;

    for (int i = 0; i < 28; i++) {
        leftHalf[27 - i] = permutedKey[55 - i];
        rightHalf[27 - i] = permutedKey[27 - i];
    }

    // 3. 生成16轮子密钥
    for (int round = 0; round < 16; round++) {
        // 循环左移
        leftHalf = leftShift(leftHalf, SHIFT_BITS[round]);
        rightHalf = leftShift(rightHalf, SHIFT_BITS[round]);

        // 合并左右两半
        bitset<56> combined;
        for (int i = 0; i < 28; i++) {
            combined[55 - i] = leftHalf[27 - i];
            combined[27 - i] = rightHalf[27 - i];
        }

        // PC-2置换：56位 -> 48位
        subKeys[round] = pc2Permutation(combined);
    }

    return subKeys;
}
```

在执行完这个程序后，我们就获得了一个长为16的bitset<48>向量，这里面存储着接下来16轮Feistel置换所需要的密钥

4.轮函数F

```
// 扩展置换函数（内部使用）
bitset<48> expansionPermutation(const bitset<32>& input) {
    bitset<48> output;

    for (int i = 0; i < 48; i++) {
        int sourceBitIndex = 32 - E[i];
        output[47 - i] = input[sourceBitIndex];
    }

    return output;
}

// S盒替换函数（内部使用）
bitset<32> sBoxSubstitution(const bitset<48>& input) {
    bitset<32> output;

    for (int i = 0; i < 8; i++) {
        int groupStart = 48 - (i + 1) * 6;
        bitset<6> sixBits;
        for (int j = 0; j < 6; j++) {
            sixBits[5 - j] = input[groupStart + j];
        }

        int row = (sixBits[5] << 1) | sixBits[0];
        int col = (sixBits[4] << 3) | (sixBits[3] << 2) | (sixBits[2] << 1) |
sixBits[1];

        int sboxValue = S_BOX[i][row][col];

        int outputStart = 32 - (i + 1) * 4;
        for (int j = 0; j < 4; j++) {
            output[outputStart + (3 - j)] = (sboxValue >> j) & 1;
        }
    }

    return output;
}

// P盒置换函数（内部使用）
bitset<32> pPermutation(const bitset<32>& input) {
    bitset<32> output;

    for (int i = 0; i < 32; i++) {
        int sourceBitIndex = 32 - P[i];
        output[31 - i] = input[sourceBitIndex];
    }
```

```
    return output;
}

// F函数
bitset<32> feistelFunction(const bitset<32>& right, const bitset<48>& subKey) {
    bitset<48> expandedRight = expansionPermutation(right);
    bitset<48> xorResult = expandedRight ^ subKey;
    bitset<32> sboxResult = sBoxSubstitution(xorResult);
    bitset<32> output = pPermutation(sboxResult);

    return output;
}
```

5.逆IP盒置换

```
bitset<64> ipInvPermutation(const bitset<64>& input) {
    bitset<64> output;

    // 执行逆IP置换
    for (int i = 0; i < 64; i++) {
        // IP_INV[i]的值是从1开始的，对应输入的第IP_INV[i]位
        // 在bitset中，第1位对应索引63
        int sourceBitIndex = 64 - IP_INV[i];
        output[63 - i] = input[sourceBitIndex];
    }

    return output;
}
```

6.一组8字节的DES加密函数

```
bitset<64> desEncryptBlock(const bitset<64>& plaintext, const bitset<64>& key) {
    // 1. 生成16轮子密钥
    vector<bitset<48>> subKeys = generateSubKeys(key);

    // 2. 初始IP置换
    bitset<64> current = ipPermutation(plaintext);

    // 3. 执行16轮Feistel变换
    for (int round = 0; round < 16; round++) {
        current = feistelRound(current, subKeys[round]);
    }

    // 4. 最后一轮后交换左右两半（因为标准DES最后一轮后不交换）
    bitset<32> left, right;
    for (int i = 0; i < 32; i++) {
        left[31 - i] = current[63 - i];
        right[31 - i] = current[31 - i];
    }
```

```
    // 交换左右
    bitset<64> swapped;
    for (int i = 0; i < 32; i++) {
        swapped[63 - i] = right[31 - i];
        swapped[31 - i] = left[31 - i];
    }

    // 5. 逆IP置换
    bitset<64> ciphertext = ipInvPermutation(swapped);

    return ciphertext;
}
```

7.对字符串的DES加密函数

```
string desEncryptString(const string& plaintext, const string& key) {
    // 1. 确保密钥是8字节
    string paddedKey = key;
    if (paddedKey.length() < 8) {
        // 如果密钥不足8字节，用0填充
        paddedKey.append(8 - paddedKey.length(), '\0');
    }
    else if (paddedKey.length() > 8) {
        // 如果密钥超过8字节，只取前8字节
        paddedKey = paddedKey.substr(0, 8);
    }

    bitset<64> keyBits = stringToBitset(paddedKey);

    // 2. PKCS#7填充
    string paddedPlaintext = pkcs7Pad(plaintext);

    // 3. 分块加密
    string ciphertext;
    for (size_t i = 0; i < paddedPlaintext.length(); i += 8) {
        string block = paddedPlaintext.substr(i, 8);
        bitset<64> plaintextBits = stringToBitset(block);
        bitset<64> ciphertextBits = desEncryptBlock(plaintextBits, keyBits);
        ciphertext += bitsetToString(ciphertextBits);
    }

    return ciphertext;
}
```

这个函数的作用是将string类型的消息进行PKCS#7填充，并分割成8字节的小块进行加密。

8.解密函数

```
bitset<64> desDecryptBlock(const bitset<64>& ciphertext, const bitset<64>& key) {
    // 1. 生成16轮子密钥
    vector<bitset<48>> subKeys = generateSubKeys(key);

    // 2. 初始IP置换
    bitset<64> current = ipPermutation(ciphertext);

    // 3. 执行16轮Feistel变换（使用逆序子密钥）
    for (int round = 0; round < 16; round++) {
        // 解密时使用逆序的子密钥
        current = feistelRound(current, subKeys[15 - round]);
    }

    // 4. 最后一轮后交换左右两半
    bitset<32> left, right;
    for (int i = 0; i < 32; i++) {
        left[31 - i] = current[63 - i];
        right[31 - i] = current[31 - i];
    }

    // 交换左右
    bitset<64> swapped;
    for (int i = 0; i < 32; i++) {
        swapped[63 - i] = right[31 - i];
        swapped[31 - i] = left[31 - i];
    }

    // 5. 逆IP置换
    bitset<64> plaintext = ipInvPermutation(swapped);

    return plaintext;
}

string desDecryptString(const string& ciphertext, const string& key) {
    // 1. 确保密钥是8字节
    string paddedKey = key;
    if (paddedKey.length() < 8) {
        paddedKey.append(8 - paddedKey.length(), '\0');
    }
    else if (paddedKey.length() > 8) {
        paddedKey = paddedKey.substr(0, 8);
    }

    bitset<64> keyBits = stringToBitset(paddedKey);

    // 2. 检查密文长度是否是8的倍数
    if (ciphertext.length() % 8 != 0) {
        cerr << "错误：密文长度不是8的倍数" << endl;
        return "";
    }

    // 3. 分块解密
    string decrypted;
```

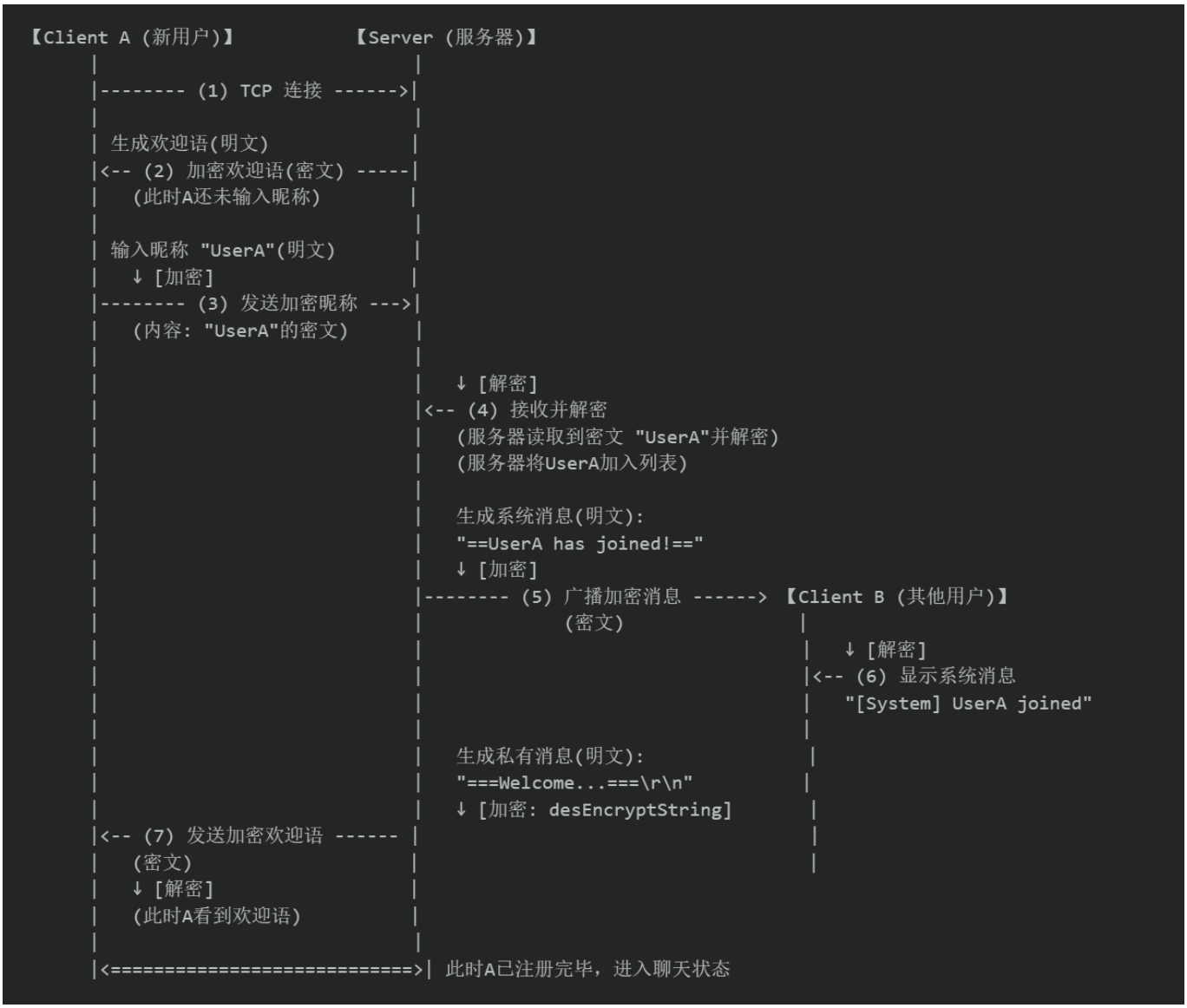
```
    for (size_t i = 0; i < ciphertext.length(); i += 8) {
        string block = ciphertext.substr(i, 8);
        bitset<64> ciphertextBits = stringToBitset(block);
        bitset<64> plaintextBits = desDecryptBlock(ciphertextBits, keyBits);
        decrypted += bitsetToString(plaintextBits);
    }

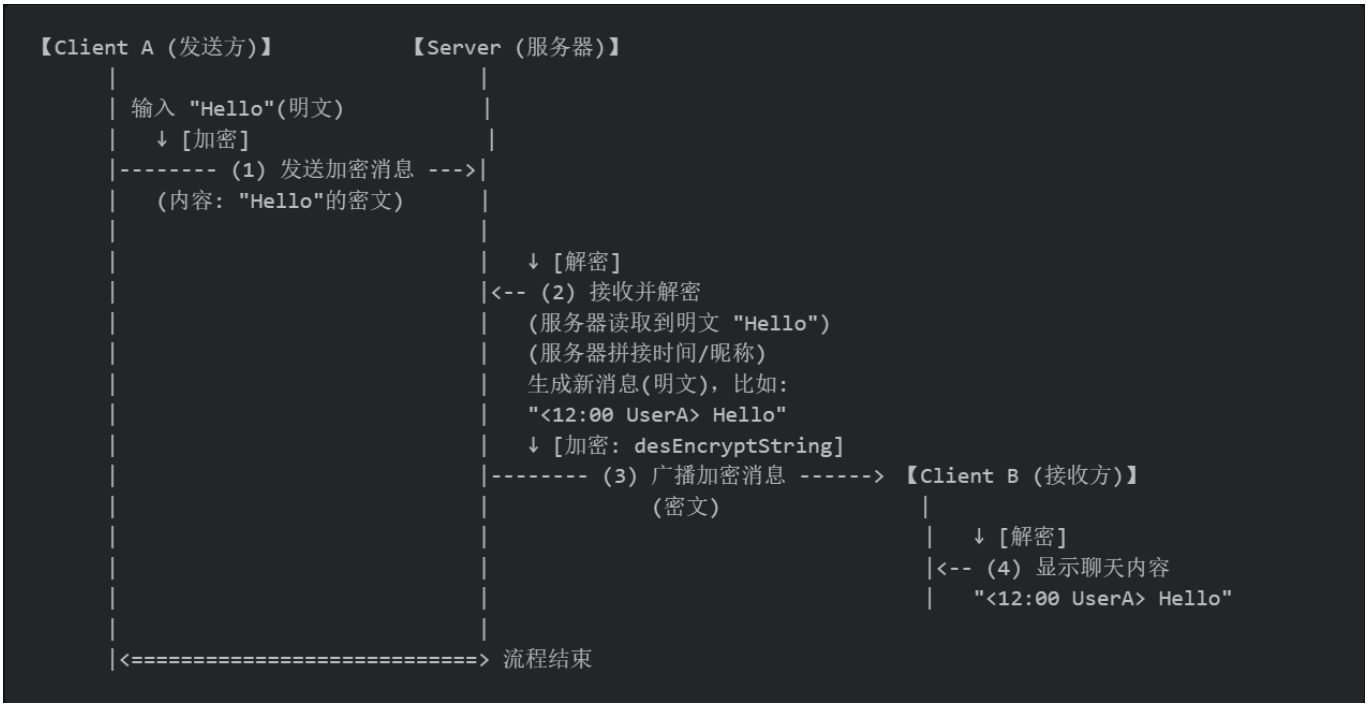
    // 4. 去除PKCS#7填充
    return pkcs7Unpad(decrypted);
}
```

基于DES加解密的TCP聊天程序

本次实验的聊天室程序源码来自于我上个学期学习《计算机网络》这个课程时的第一次作业，也是一个TCP聊天室程序，只是没有DES加解密，相关源码、报告、可执行文件已经提交到计算机学院官网上，这里不再赘述具体的聊天室程序代码，只说明在哪些关键的地方进行了加密和解密，如有兴趣，聊天室的实现可以查看与本次作业一并提交的计网实验报告.pdf文件。

总体的过程大概是这样的：





实验结果

关于DES加解密的验证

测试程序：

```
int main() {
    // 8字节明文: "12345678"
    string plaintext = "ABCDEFGH";
    // 8字节密钥: "12345678"
    string key = "12345678";

    cout << "========== DES encryption test ==========" << endl << endl;

    // encrypt
    string ciphertext = desEncryptString(plaintext, key);

    // decrypt
    string decrypted = desDecryptString(ciphertext, key);

    // output plaintext (hex)
    cout << "Plaintext: ";
    for (int i = 0; i < 8; i++) {
        cout << hex << setw(2) << setfill('0') << (int)(unsigned
char)plaintext[i];
        if (i < 7) cout << " ";
    }
    cout << dec << endl;

    // output plaintext (binary)
    cout << "Plaintext binary: ";
    bitset<64> plainBits = stringToBitset(plaintext);
    for (int i = 0; i < 64; i++) {
        cout << plainBits[63 - i];
        if ((i + 1) % 8 == 0 && i < 63) cout << " ";
    }
    cout << endl << endl;

    // output ciphertext (hex)
    cout << "Ciphertext: ";
    for (int i = 0; i < 8; i++) {
        cout << hex << setw(2) << setfill('0') << (int)(unsigned
char)ciphertext[i];
        if (i < 7) cout << " ";
    }
    cout << dec << endl;

    // output ciphertext (binary)
    cout << "Ciphertext binary: ";
    bitset<64> cipherBits = stringToBitset(ciphertext);
    for (int i = 0; i < 64; i++) {
        cout << cipherBits[63 - i];
        if ((i + 1) % 8 == 0 && i < 63) cout << " ";
    }
    cout << endl << endl;

    // output decrypted plaintext
```

```
    cout << "Decrypted: " << decrypted << endl;
    cout << "Decrypted hex: ";
    for (int i = 0; i < 8; i++) {
        cout << hex << setw(2) << setfill('0') << (int)(unsigned
char)decrypted[i];
        if (i < 7) cout << " ";
    }
    cout << dec << endl;

    // verify
    cout << endl << "Verification: " << (plaintext == decrypted ? "PASS" : "FAIL")
<< endl;

    return 0;
}
```

输出结果：

```
========== DES encryption test ==========

Plaintext: 41 42 43 44 45 46 47 48
Plaintext binary: 01000001 01000010 01000011 01000100 01000101 01000110 01000111
01001000

Ciphertext: 3c f7 31 be 49 92 21 c9
Ciphertext binary: 00111100 11110111 00110001 10111110 01001001 10010010 00100001
11001001

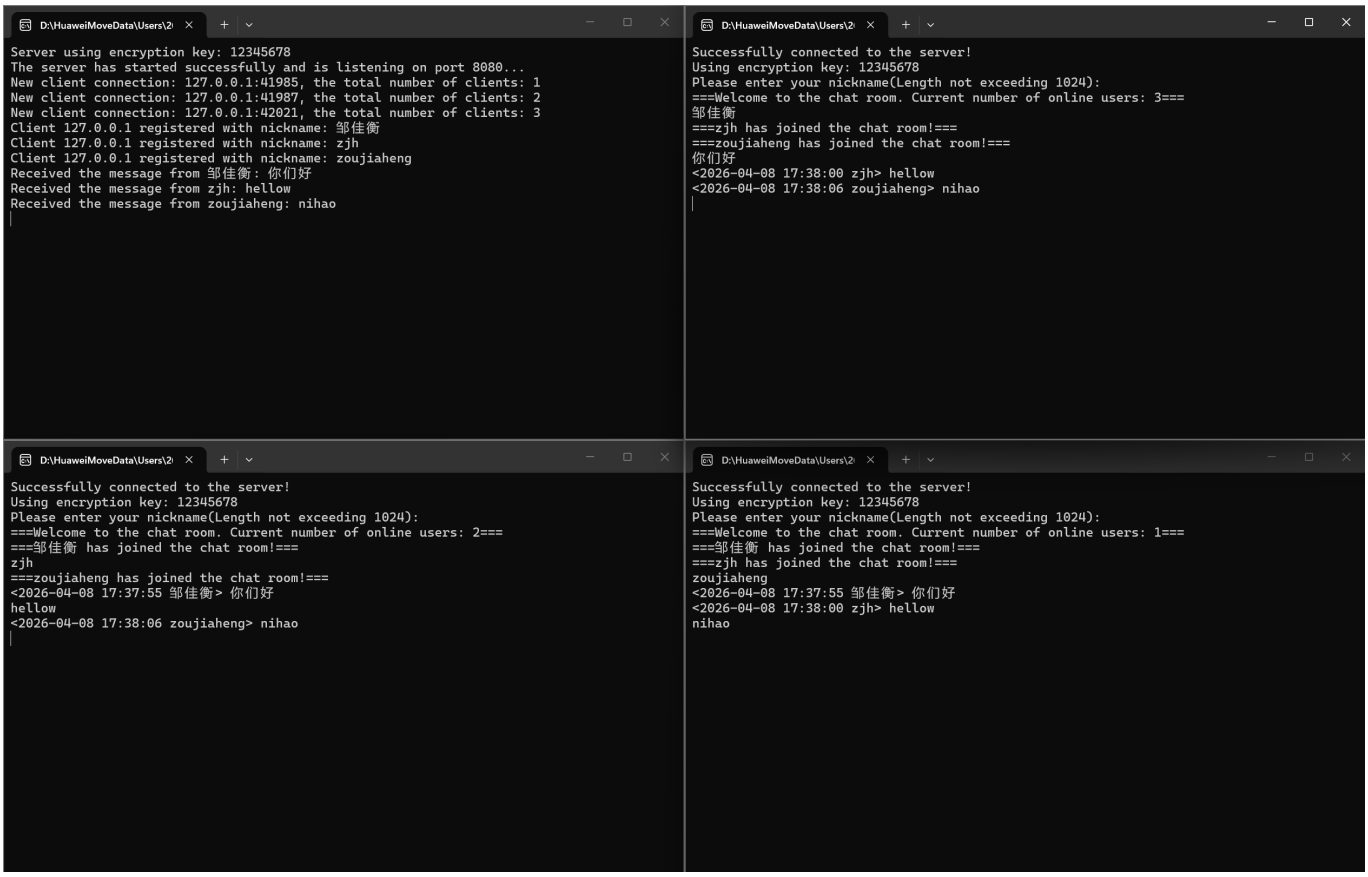
Decrypted: ABCDEFGH
Decrypted hex: 41 42 43 44 45 46 47 48

Verification: PASS
```

DES加解密程序验证成功。

聊天室测试

具体请查看聊天室.mp4，这里只放最终图片。



实验总结与反思

本次实验成功实现了一个基于DES对称加密算法的安全聊天系统。通过编程实践，我深入理解了DES算法的加解密流程，以及其在网络通信中的具体应用。实验中，我完成了客户端与服务器的Socket通信调整，并实现了“发送前加密、接收后解密”的安全机制，确保了聊天内容（包括昵称和消息）在传输过程中的机密性。测试结果表明，系统能够准确完成加解密操作，通信稳定。此次实验不仅提升了我的C++网络编程能力，更让我直观体会到了密码学在保障数据安全方面的重要作用。

关于本项目的未来的改进，目前，本项目只能在编程文件DES.cpp里设置密钥常量，未来如果能够生成随机密钥，安全性会大大增加，另外，本项目在客户端和服务端里分别存在两份DES加密的相关编程文件，客户端及服务端与DES加密文件的引用方式较为原始，应该可以封装成静态或动态库，此外，客户端、服务端有一些地方存在解密后又重新加密的情况，如果能适当调整代码顺序，应该能有更高的执行效率，子密钥的生成也存在重复计算的问题，这些都是可以继续改进的空间。